

Relazione tecnica

M2_S4_E2 - GameShell e verifica controllata di autenticazione SSH

D'Angelo Giovanni

Attività svolta in ambiente Kali Linux con focus su familiarizzazione con la shell, progressione nei livelli di GameShell e predisposizione di un laboratorio SSH per test controllati di autenticazione.

1. Obiettivo dell'attività

L'attività aveva due obiettivi distinti ma collegati tra loro. Il primo consisteva nell'utilizzare GameShell per prendere confidenza con la shell Linux, con i comandi di navigazione del filesystem e con le operazioni di lettura, ricerca e gestione dei file. Il secondo consisteva nel predisporre un laboratorio SSH e nel realizzare un programma Python per provare combinazioni username/password esclusivamente in un contesto di test autorizzato.

Ho quindi organizzato il lavoro in due parti: nella prima mi sono concentrato sull'installazione del gioco e sul completamento di più missioni possibili; nella seconda ho verificato il servizio SSH, ho preparato utente e file di supporto e ho strutturato la directory del progetto per raccogliere script, wordlist e risultati.

2. Ambiente di lavoro

Elemento	Dettaglio
Sistema operativo	Kali Linux in macchina virtuale
Task 1	Installazione e utilizzo di GameShell per esercitarsi con la shell Linux
Task 2	Predisposizione di un laboratorio SSH e organizzazione del tester Python
Comandi e strumenti ricorrenti	apt, wget, bash, gsh, ls, cd, pwd, nano, service, ssh, ip a
Materiale prodotto	Script Python, file utenti/password, struttura del progetto e documentazione operativa

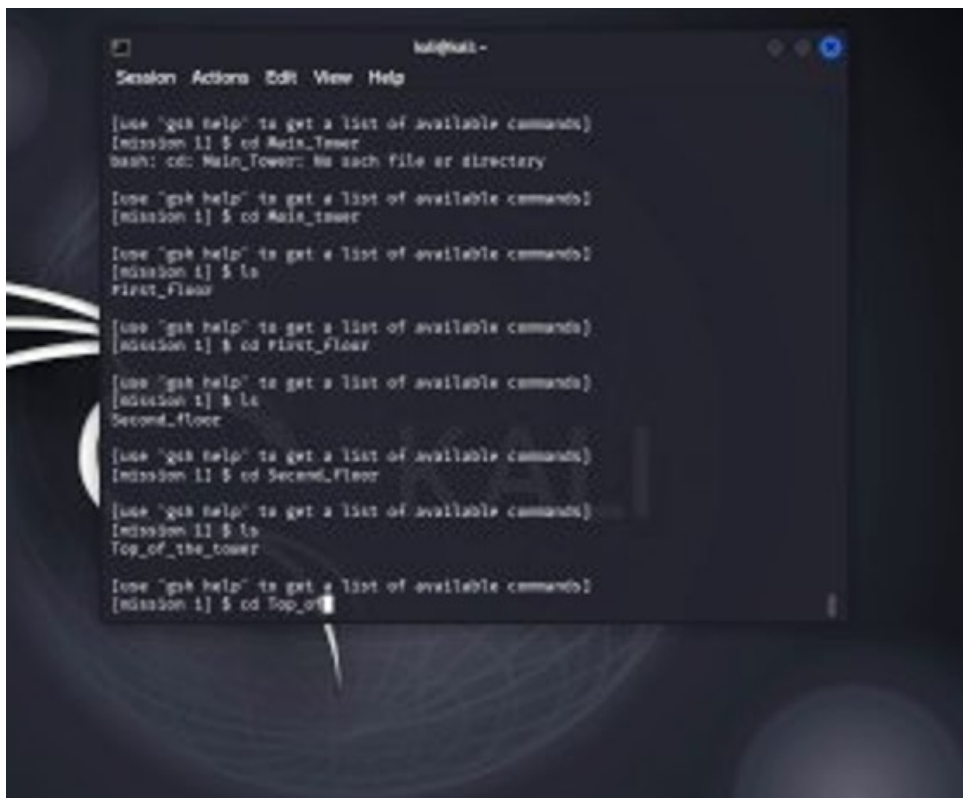
3. Task 1 - Installazione e utilizzo di GameShell

Per prima cosa ho aggiornato l'ambiente e ho preparato i pacchetti necessari all'installazione del gioco. In questa fase l'obiettivo era disporre di tutti gli strumenti utili alla gestione della shell e dei file, in modo da poter eseguire correttamente le missioni previste.

Dopo l'aggiornamento del sistema ho installato i pacchetti richiesti e ho scaricato lo script di GameShell. Terminato il download, ho avviato il gioco direttamente da terminale e sono entrato nell'ambiente a missioni.

```
sudo apt update
sudo apt install gettext man-db procps psmisc nano tree bsdmainutils x11-apps wget
wget https://github.com/phyver/GameShell/releases/download/latest/gameshell.sh
bash gameshell.sh
```

Una volta aperto GameShell ho utilizzato in modo sistematico i comandi gsh goal e gsh check per leggere l'obiettivo corrente e verificare il completamento della missione. Parallelamente ho fatto uso dei comandi Linux suggeriti dal gioco per orientarmi nelle cartelle e portare a termine i passaggi richiesti.



```

[use "gsh help" to get a list of available commands]
Mission 1] $ cd Main_Tower
bash: cd: Main_Tower: No such file or directory

[use "gsh help" to get a list of available commands]
[Mission 1] $ cd Main_tower

[use "gsh help" to get a list of available commands]
[Mission 1] $ ls
First_Floor

[use "gsh help" to get a list of available commands]
[Mission 1] $ cd First_Floor

[use "gsh help" to get a list of available commands]
[Mission 1] $ ls
Second_Floor

[use "gsh help" to get a list of available commands]
[Mission 1] $ cd Second_Floor

[use "gsh help" to get a list of available commands]
[Mission 1] $ ls
Top_of_the_tower

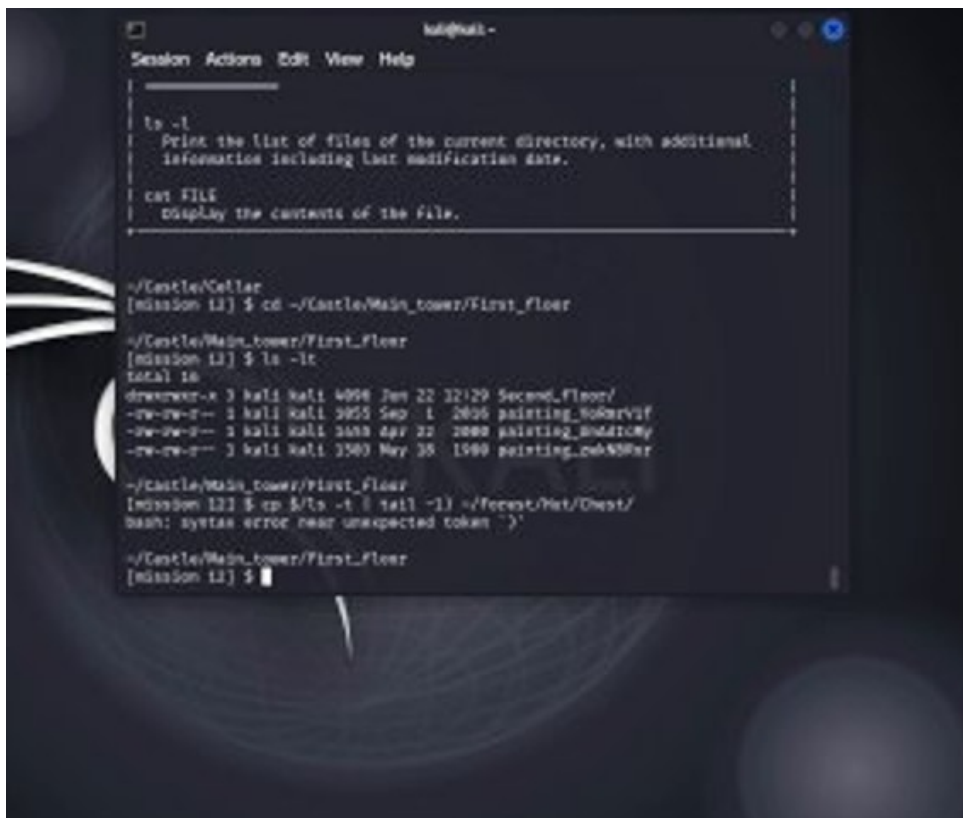
[use "gsh help" to get a list of available commands]
[Mission 1] $ cd Top_of

```

Figura 1 - Progressione iniziale in GameShell fino alla torre principale del castello.

Nella fase iniziale ho completato la navigazione verso la torre principale del castello usando pwd, ls e diversi comandi cd, spostandomi tra Castle, Main_tower, First_floor, Second_floor e Top_of_the_tower. Questo passaggio mi ha permesso di verificare concretamente come leggere il path corrente, elencare il contenuto di una directory e muovermi in profondità nel filesystem.

Proseguendo con le missioni successive ho affrontato richieste sempre più varie: consultazione del contenuto di file, uso di percorsi relativi, spostamento di elementi e verifica dell'ordinamento delle informazioni. In questa parte del lavoro ho consolidato soprattutto i comandi cat, ls con opzioni aggiuntive e alcune operazioni di ricerca e filtro.



```
Session Actions Edit View Help
-----
ls -l
Print the list of files of the current directory, with additional
information including last modification date.
cat FILE
Display the contents of the file.

~/Castle/Cellar
[mission 12] $ cd ~/Castle/Main_tower/First_floor
~/Castle/Main_tower/First_floor
[mission 12] $ ls -l
total 16
drwxrwxr-x 3 kali kali 4096 Jan 22 12:29 Second_Floor/
-rw-rw-r-- 1 kali kali 3925 Sep  1  2006 painting_HoRervVf
-rw-rw-r-- 1 kali kali 3488 Apr 22  2000 painting_Boddicmy
-rw-rw-r-- 1 kali kali 3583 May 25  1999 painting_cobN8Wkr

~/Castle/Main_tower/First_floor
[mission 12] $ cp $/ls -l | tail -1 ~/Forest/Net/Onest/
bash: syntax error near unexpected token `)'

~/Castle/Main_tower/First_floor
[mission 12] $
```

Figura 2 - Esempio di missione avanzata con elenco dettagliato e ordinato dei file.

Dalla progressione registrata emerge che non mi sono fermato alla sola missione introduttiva. Ho infatti portato avanti il gioco fino a livelli più avanzati, nei quali era necessario passare dalla semplice navigazione al ragionamento sui file presenti, sul loro contenuto e sulla forma dell'output richiesto. Questo aspetto è stato utile perché mi ha costretto a usare i comandi non solo in modo meccanico, ma anche nel formato corretto richiesto dalla missione.

In sintesi, il Task 1 mi ha permesso di trasformare la teoria della shell in operatività reale: ho letto gli obiettivi, ho individuato i comandi adatti, ho verificato passo dopo passo lo stato della missione e ho consolidato una metodologia ordinata da terminale.

4. Task 2 - Predisposizione del laboratorio SSH

La seconda parte del progetto è stata dedicata alla predisposizione di un ambiente SSH di laboratorio. L'obiettivo non era utilizzare un servizio esterno, ma verificare il funzionamento del servizio su una macchina controllata, creare le credenziali di test e organizzare i file necessari all'esecuzione del programma Python.

In questa fase ho verificato lo stato del servizio SSH su Kali, avviandolo e controllandone il corretto funzionamento. Il controllo del servizio mi ha confermato che OpenSSH era attivo e in ascolto.

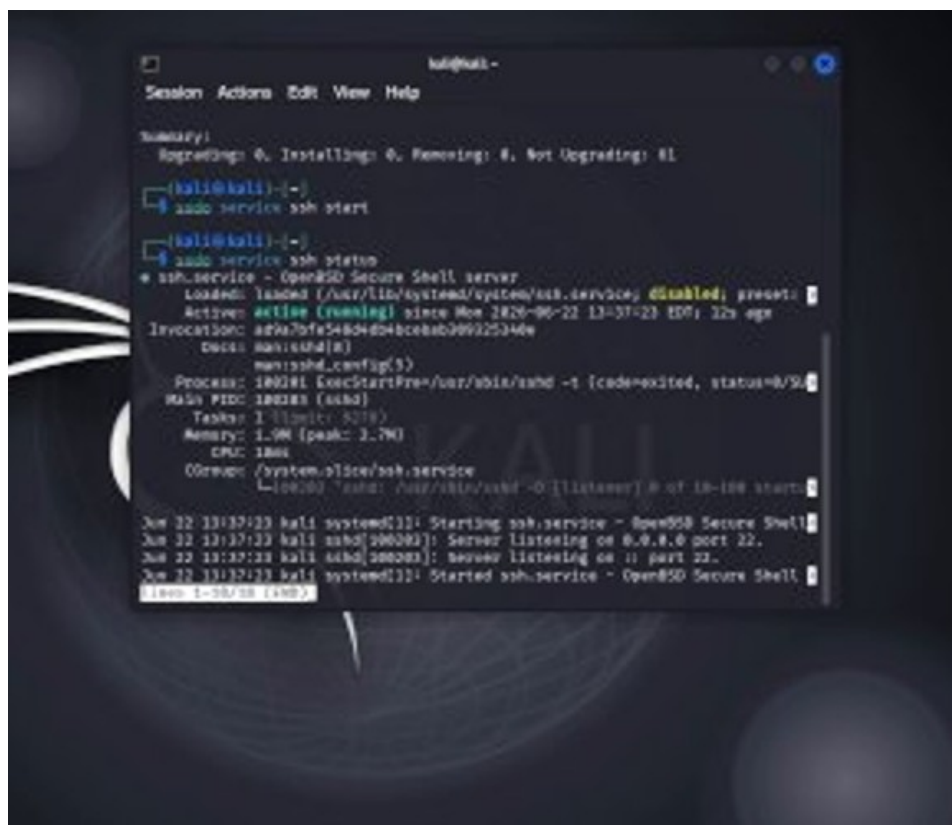
A screenshot of a Kali Linux terminal window. The window title is 'kali@kali -'. The menu bar shows 'Session', 'Actions', 'Edit', 'View', and 'Help'. The terminal output shows the following commands and their results:
1. Command: `sudo service ssh start`
2. Command: `sudo service ssh status`
Output for status command:
* ssh.service - OpenSSH Secure Shell server
Loaded: loaded (/usr/lib/systemd/system/ssh.service; disabled; preset: enabled)
Active: active (running) since Mon 2020-06-22 13:37:23 EDT; 12s ago
Invocation: a89a7bfe548d4db4bceeb30992534de
Process: 180281 ExecStartPre=/usr/sbin/sshd -t [code=exited, status=0/SUCCESS]
Main PID: 180283 (sshd)
Tasks: 1 (limit: 512)
Memory: 1.9M (peak: 3.7M)
CPU: 18ms
CGroup: /system.slice/ssh.service
Jun 22 13:37:23 kali systemd[1]: Starting ssh.service - OpenSSH Secure Shell server:
Jun 22 13:37:23 kali sshd[180283]: Server listening on 0.0.0.0 port 22.
Jun 22 13:37:23 kali sshd[180283]: Server listening on *: port 22.
Jun 22 13:37:23 kali systemd[1]: Started ssh.service - OpenSSH Secure Shell server:
The terminal also shows a prompt `root@kali:~#` at the bottom.

Figura 3 - Avvio e verifica del servizio OpenSSH su Kali Linux.

Successivamente ho creato o verificato l'utente di laboratorio dedicato al test, ho controllato i dati dell'account con id e ho raccolto le informazioni di rete con ip a. Questa verifica è stata importante perché mi ha consentito di sapere su quale interfaccia e su quale indirizzo IP operare.

Prima di procedere con l'automazione ho effettuato una prova manuale di accesso SSH verso localhost. In questo modo ho confermato che l'utente di test era effettivamente in grado di autenticarsi e che il servizio rispondeva correttamente sulla macchina di laboratorio.

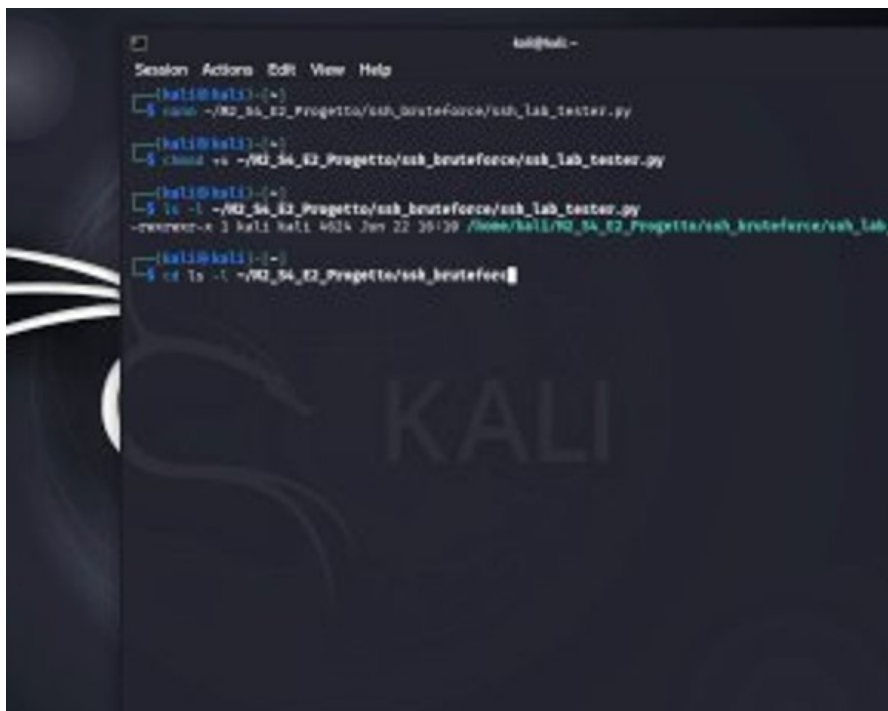


Figura 4 - Test manuale di accesso SSH verso localhost con l'utente di laboratorio.

5. Preparazione dello script e dei file di supporto

Dopo aver validato il servizio ho lavorato sulla struttura del progetto. Ho organizzato una cartella dedicata con sottocartelle separate per GameShell, brute force SSH, screenshot e report, così da tenere distinti i componenti del progetto e facilitare il recupero dei file.

All'interno della directory ssh_bruteforce ho predisposto i file di supporto, in particolare la lista utenti, la lista password e lo script Python. La presenza di questi file mi ha consentito di avere una struttura chiara e ripetibile, evitando di lasciare comandi e materiale sparsi nel filesystem.



```
kali@kali:~$  
Session Actions Edit View Help  
[kali@kali]~$  
$ nano ~/M2_S4_E2_Progetto/ssh_bruteforce/ssh_lab_tester.py  
[kali@kali]~$  
$ chmod +x ~/M2_S4_E2_Progetto/ssh_bruteforce/ssh_lab_tester.py  
[kali@kali]~$  
$ ls -l ~/M2_S4_E2_Progetto/ssh_bruteforce/ssh_lab_tester.py  
-rwxr-xr-x 1 kali kali 4014 Jan 22 20:29 /home/kali/M2_S4_E2_Progetto/ssh_bruteforce/ssh_lab_tester.py  
[kali@kali]~$  
$ cd /home/kali/M2_S4_E2_Progetto/ssh_bruteforce/
```

Figura 5 - Verifica della presenza dello script Python all'interno della cartella di progetto.

Nel corso della preparazione ho inoltre creato e salvato i file di testo con le credenziali di prova. La lista password conteneva combinazioni semplici e una password di laboratorio valida, mentre il file utenti raccoglieva gli account da provare nel contesto controllato. Questa organizzazione mi ha consentito di separare con chiarezza la logica dello script dai dati usati per il test.

6. Esecuzione del tester e risultati osservati

L'esecuzione automatizzata è avvenuta in più fasi. In un primo momento ho lanciato il tester verso un indirizzo IP privato di laboratorio, dopo aver effettuato i controlli di rete di base. Il risultato di questa prova non è stato positivo: dai tentativi registrati risulta infatti che sulla porta 22 del target non era disponibile un servizio SSH raggiungibile, per cui il programma ha restituito solo errori di connessione e nessuna credenziale valida trovata.



```
kali@kali: ~/M2_S4_E2/ProgettoLab_bruteforce
on Actions Edit View Help
errore connessione con ssh_lab:admin -> [Errore Nones] Unable to connect to port 22 on
127.0.0.1
entratina 18: ssh_lab:LabPass123
errore connessione con ssh_lab:LabPass123 -> [Errore Nones] Unable to connect to port
127.0.0.1
entratina 19: admin:password
errore connessione con admin:password -> [Errore Nones] Unable to connect to port 22 o
n 127.0.0.1
entratina 20: admin:113456
errore connessione con admin:113456 -> [Errore Nones] Unable to connect to port 22 o
n 127.0.0.1
entratina 21: admin:kali
errore connessione con admin:kali -> [Errore Nones] Unable to connect to port 22 on 12
7.0.0.1
entratina 22: admin:epicode
errore connessione con admin:epicode -> [Errore Nones] Unable to connect to port 22 on
127.0.0.1
entratina 23: admin:admin
errore connessione con admin:admin -> [Errore Nones] Unable to connect to port 22 on 1
27.0.0.1
entratina 24: admin:LabPass123
errore connessione con admin:LabPass123 -> [Errore Nones] Unable to connect to port 22
on 127.0.0.1
nessuna credenziale valida trovata.
kali@kali:~/M2_S4_E2/ProgettoLab_bruteforce:
```

Figura 6 - Primo test automatico verso target privato con porta 22 non raggiungibile.

Questo passaggio è stato comunque utile, perché ha mostrato che la sola esistenza di un indirizzo IP raggiungibile non è sufficiente: per completare il test servono un servizio attivo, una porta aperta e credenziali coerenti con il sistema bersaglio. In altre parole, l'errore non dipendeva dalla sintassi del comando, ma dalla disponibilità effettiva del servizio remoto.

Per rendere il laboratorio coerente con i requisiti dell'esercizio ho quindi consolidato la verifica locale su Kali: ho avviato il servizio SSH, ho testato l'accesso manuale a 127.0.0.1, ho predisposto utente e file di supporto e ho sistemato la struttura del progetto. Questa seconda fase mi ha lasciato un ambiente pronto e ordinato per il test controllato nel contesto locale.

Dal punto di vista operativo, il lavoro svolto nel Task 2 ha avuto quindi un doppio valore: da una parte mi ha mostrato l'importanza di verificare preliminarmente servizio, porta e credenziali; dall'altra mi ha permesso di costruire correttamente tutta l'infrastruttura minima necessaria per l'esercizio.

7. Comandi principali utilizzati

Task 1

sudo apt update

sudo apt install gettext man-db procps psmisc nano tree bsdmainutils x11-apps wget

wget <https://github.com/phyver/GameShell/releases/download/latest/gameshell.sh>

bash gameshell.sh

gsh goal

pwd

ls

cd ...


```
gsh check
```

```
# Task 2
```

```
sudo service ssh start
```

```
sudo service ssh status
```

```
id ssh_lab
```

```
ip a
```

```
ssh ssh_lab@127.0.0.1
```

```
mkdir -p ~/M2_S4_E2_Progetto/{gameshell,ssh_bruteforce,screenshot,report}
```

```
nano password_test.txt
```

```
nano users_test.txt
```

8. Considerazioni tecniche finali

L'attività mi ha permesso di lavorare sia sul lato sistemistico sia sul lato applicativo. Con GameShell ho rafforzato l'uso pratico della shell Linux, imparando a leggere con attenzione il contesto della missione e a scegliere il comando più adatto in base all'obiettivo. Questo mi ha aiutato a migliorare la gestione di directory, file, contenuti e output da terminale.

Nel laboratorio SSH ho verificato l'importanza dei controlli preliminari: stato del servizio, apertura della porta, correttezza dell'utente di test e validità delle credenziali. Il primo tentativo verso un target privato non ha restituito un risultato utile proprio perché mancava una disponibilità effettiva del servizio remoto sulla porta attesa.

La successiva validazione locale su Kali ha invece confermato che, quando il servizio è attivo e l'utente è configurato correttamente, il contesto di prova è coerente con quanto richiesto dall'esercizio. Questo approccio mi ha consentito di chiudere il progetto con una struttura ordinata e con un ambiente di lavoro già pronto per ulteriori verifiche e documentazione.

9. Conclusione

Nel complesso ho completato un'attività articolata in due parti complementari. Da un lato ho utilizzato GameShell per mettere in pratica i comandi Linux fondamentali e avanzare attraverso più missioni, dall'altro ho predisposto un laboratorio SSH controllato su Kali, verificando il servizio, l'utente di test e la struttura dei file necessari allo script Python. Il progetto mi ha permesso di consolidare metodo, precisione operativa e capacità di controllo dell'ambiente prima dell'esecuzione di qualunque test.